Análisis exploratorio: hábitos de escucha musical por país (Last.fm, 2005–2014)

Objetivo

Conocer y preparar los datos de escuchas de Last.fm para analizar cómo se diferencian los hábitos de escucha musical entre países.

Datos

- **Culture-Aware Music Recommendation Dataset** (Zangerle, Pichl y Schedl, 2020), subconjunto del dataset LFM-1b de Last.fm. Fuente: https://zenodo.org/records/3477842
- **Nombres de artistas** del LFM-1b. Fuente: https://zenodo.org/record/3964506

Los archivos originales (.tsv) fueron convertidos previamente a formato parquet con el script `paso2_a_parquet.py` .

Herramientas

- **DuckDB**: para consultar los datos con SQL. Se eligió porque la tabla de escuchas tiene 351 millones de filas y no entra en memoria con pandas.
- **pandas**: para manejar los resultados de las consultas (tablas chicas).
- **matplotlib**: para los gráficos.

1. Preparación del entorno

In [1]:
```
# --- Librerías ---
import duckdb                    # motor SQL que lee archivos parquet sin
import pandas as pd              # para trabajar con los resultados como
import matplotlib.pyplot as plt  # para graficar

# --- Conexión a DuckDB ---
# connect() sin argumentos crea una base de datos en memoria (no se guard
# Solo la usamos para consultar los parquet.
con = duckdb.connect()

# Limitamos la memoria que puede usar DuckDB.
# Si una consulta necesita más, usa el disco en vez de trabar la computad
con.execute("SET memory_limit = '4GB'")

#El timestamp del dataset está en "unix time": segundos desde el 1/1/1970
# DuckDB muestra las fechas en la zona horaria de la computadora, salvo q
# Fijamos UTC para que las fechas no dependan de dónde se ejecute el note
# y todas las horas aparecerían corridas.
con.execute("SET TimeZone = 'UTC'")
```

```
# --- Vistas: un nombre corto para cada archivo ---
# Una vista NO copia los datos: es como un "acceso directo" al archivo.
# Así en las consultas podemos escribir FROM users en vez de FROM 'data/p
ruta = "data/parquet/"

con.execute(f"CREATE OR REPLACE VIEW events            AS SELECT * FROM '
con.execute(f"CREATE OR REPLACE VIEW users             AS SELECT * FROM '
con.execute(f"CREATE OR REPLACE VIEW artists           AS SELECT * FROM '
con.execute(f"CREATE OR REPLACE VIEW acoustic_features AS SELECT * FROM '
con.execute(f"CREATE OR REPLACE VIEW hofstede          AS SELECT * FROM '
con.execute(f"CREATE OR REPLACE VIEW happiness         AS SELECT * FROM '

# --- Función auxiliar ---
# Recibe una consulta SQL (como texto), la ejecuta y devuelve el resultad
# La usamos para no repetir con.sql(...).df() en cada celda.
def q(consulta):
    return con.sql(consulta).df()

# Si se ve la tabla con "todo listo" la conexión funciona.
q("SELECT 'todo listo' AS estado")
```

Out[1]:

	estado
0	todo listo

2. Conocer y explorar los datos

Antes de analizar, revisamos cada tabla para entender qué contiene:

- **2.1** Cuántas filas tiene cada tabla,
- **2.2** Qué columnas tiene y de qué tipo es cada una,
- **2.3** Cómo se ven las primeras filas.
- **2.4** Granularidad de entidades y modelo de relación (ERD)
- **2.5** Cobertura, distribuciones demográficas y patrón temporal
- **2.6** Síntesis de la abstracción y características del dataset

2.1. Filas por tabla

In [2]:
```
# Lista con los nombres de las vistas que creamos en el paso 1
tablas = ["events", "users", "artists", "acoustic_features", "hofstede",

# Para cada tabla contamos las filas con COUNT(*).
# fetchone()[0] devuelve directamente el número (en vez de una tabla de u
for tabla in tablas:
    filas = con.execute(f"SELECT COUNT(*) FROM {tabla}").fetchone()[0]
    print(f"{tabla:20s} {filas:>15,} filas")   # :20s y :>15, solo alinea
```

```
events                351,469,333 filas
users                      55,190 filas
artists                 3,190,371 filas
acoustic_features       3,471,884 filas
hofstede                       47 filas
happiness                   1,562 filas
```

2.2. Columnas de cada tabla

In [3]:
```
# DESCRIBE muestra cada columna de la tabla con su tipo de dato
# equivalente de df.info() en pandas.
# display() muestra cada resultado como tabla dentro del notebook.
for tabla in tablas:
    print(f"--- {tabla} ---")
    df_desc = con.execute(f"DESCRIBE {tabla}").df()
    display(df_desc[["column_name", "column_type"]])  # nos quedamos solo
```

```
--- events ---
```

	column_name	column_type
0	user_id	INTEGER
1	artist_id	INTEGER
2	album_id	INTEGER
3	track_id	INTEGER
4	timestamp	BIGINT

```
--- users ---
```

	column_name	column_type
0	user_id	BIGINT
1	country	VARCHAR
2	age	BIGINT
3	gender	VARCHAR
4	playcount	BIGINT
5	registered_unixtime	BIGINT

```
--- artists ---
```

	column_name	column_type
0	artist_id	INTEGER
1	artist_name	VARCHAR
2	gender_counts	VARCHAR

```
--- acoustic_features ---
```

	column_name	column_type
0	track_id	BIGINT
1	danceability	DOUBLE
2	energy	DOUBLE
3	key	DOUBLE
4	loudness	DOUBLE
5	mode	DOUBLE
6	speechiness	DOUBLE
7	acousticness	DOUBLE
8	instrumentalness	DOUBLE
9	liveness	DOUBLE
10	valence	DOUBLE
11	tempo	DOUBLE

```
--- hofstede ---
```

	column_name	column_type
0	no	BIGINT
1	ctr	VARCHAR
2	country	VARCHAR
3	power_distance	VARCHAR
4	individualism	VARCHAR
5	masculinity	VARCHAR
6	uncertainty_avoidance	VARCHAR
7	long_term_orientation	BIGINT
8	indulgence	VARCHAR

```
--- happiness ---
```

	column_name	column_type
0	country	VARCHAR
1	year	BIGINT
2	Life Ladder	DOUBLE
3	Log GDP per capita	DOUBLE
4	Social support	DOUBLE
5	Healthy life expectancy at birth	DOUBLE
6	Freedom to make life choices	DOUBLE
7	Generosity	DOUBLE
8	Perceptions of corruption	DOUBLE
9	Positive affect	DOUBLE
10	Negative affect	DOUBLE
11	Confidence in national government	DOUBLE
12	Democratic Quality	DOUBLE
13	Delivery Quality	DOUBLE
14	Standard deviation of ladder by country-year	DOUBLE
15	Standard deviation/Mean of ladder by country-year	DOUBLE
16	GINI index (World Bank estimate)	DOUBLE
17	GINI index (World Bank estimate), average 2000-15	DOUBLE
18	gini of household income reported in Gallup, b...	DOUBLE

2.3. Primeras filas de cada tabla

In [4]:
```
# LIMIT 5 trae solo 5 filas
# Es el equivalente de df.head() en pandas.
for tabla in tablas:
    print(f"--- {tabla} ---")
    display(q(f"SELECT * FROM {tabla} LIMIT 5"))
```

```
--- events ---
```

	user_id	artist_id	album_id	track_id	timestamp
0	31435741	21	31	53	1370977938
1	31435741	21	31	53	1370977728
2	31435741	21	31	53	1370977518
3	31435741	21	31	53	1370977308
4	31435741	21	31	53	1370977098

```
--- users ---
```

	user_id	country	age	gender	playcount	registered_unixtime
0	384	UK	35	m	42139	1035849600
1	3653	UK	31	m	18504	1041033600
2	4813	US	43	m	640	1050364800
3	5069	AT	30	m	31867	1051488000
4	5213	UK	37	n	392	1052265600

```
--- artists ---
```

	artist_id	artist_name	gender_counts
0	1	Megadeth	4/20/0/0/0
1	2	Lil Wayne	0/1/0/0/0
2	3	Foo Fighters	0/8/0/0/0
3	4	Porcupine Tree	0/6/0/0/0
4	5	No-Man	1/3/0/0/0

```
--- acoustic_features ---
```

	track_id	danceability	energy	key	loudness	mode	speechiness	acousticness	inst
0	16075483	0.570	0.716	3.0	-5.957	0.0	0.4490	0.184000	
1	19741900	0.218	0.208	9.0	-10.370	0.0	0.0336	0.965000	
2	299887	0.533	0.400	7.0	-12.368	1.0	0.0253	0.902000	
3	7891581	0.657	0.591	6.0	-8.501	0.0	0.0563	0.045100	
4	3869192	0.545	0.720	2.0	-8.585	1.0	0.0441	0.000798	

```
--- hofstede ---
```

	no	ctr	country	power_distance	individualism	masculinity	uncertainty_avoidance
0	1	AR	Argentina	49	46	56	86
1	2	AT	Austria	11	55	79	70
2	3	AU	Australia	38	90	61	51
3	4	BE	Belgium	65	75	54	94
4	5	BG	Bulgeria	70	30	40	85

```
--- happiness ---
```

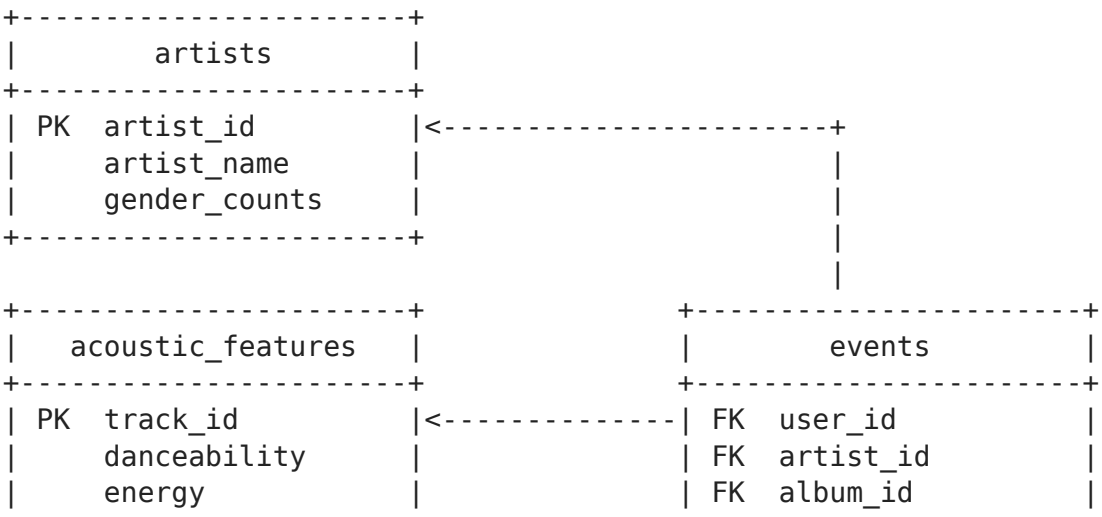
	country	year	Life Ladder	Log GDP per capita	Social support	Healthy life expectancy at birth	Freedom to make life choices	Generosity	I
0	Afghanistan	2008	3.723590	7.168690	0.450662	49.209663	0.718114	0.181819	
1	Afghanistan	2009	4.401778	7.333790	0.552308	49.624432	0.678896	0.203614	
2	Afghanistan	2010	4.758381	7.386629	0.539075	50.008961	0.600127	0.137630	
3	Afghanistan	2011	3.831719	7.415019	0.521104	50.367298	0.495901	0.175329	
4	Afghanistan	2012	3.782938	7.517126	0.520637	50.709263	0.530935	0.247159	

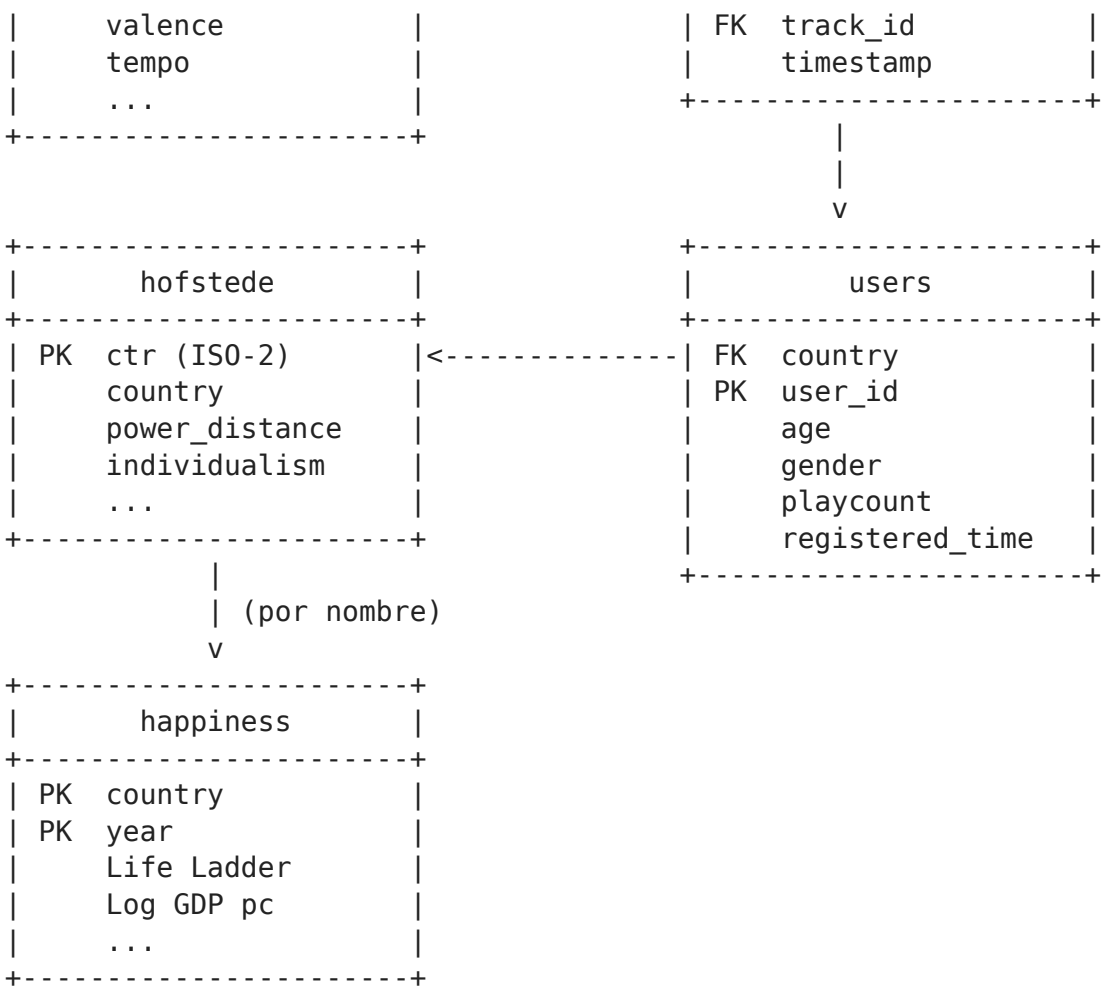
2.4 Granularidad de entidades y modelo de relación (ERD)

Para pasar de los elementos del dominio a una **abstracción de los datos**, definimos qué representa cada observación (granularidad de cada fila) y cuáles son los atributos clave (*primary/foreign keys*) utilizados para vincular las 6 tablas del dataset mediante un esquema en estrella modificado.

Resumen de Granularidad y Claves de Unión

Tabla	Una fila representa...	Se une por (Clave)	Tabla de destino y Cardinalidad
events	Una escucha individual (un usuario reprodujo una canción en un instante)	`user_id` , `artist_id` , `track_id`	Tabla central de hechos. Vincula con `users` (N:1), `artists` (N:1) y `acoustic_features` (N:1)
users	Un perfil de usuario registrado en Last.fm	`user_id` `country`	Dimensiones de usuario. Se une con `events` por `user_id` y con `hofstede` mediante `country = ctr`
artists	Un artista musical del catálogo LFM-1b	`artist_id`	Dimensión de artista. Se une con `events` por `artist_id`
acoustic_features	Una canción única con sus descriptores de audio de Spotify	`track_id`	Dimensión de audio. Se une con `events` por `track_id`
hofstede	Un país con sus dimensiones culturales	`ctr` (código ISO-2)	Dimensión cultural. Se une con `users` por `country = ctr` (mapeando `UK` → `GB`)
happiness	Un país en un año de encuesta (World Happiness Report)	`country` + `year`	Dimensión socioeconómica. Se une mediante el nombre en inglés del país





2.5 Cobertura, distribuciones demográficas y patrón temporal

En esta subsección evaluamos la cobertura del catálogo de artistas, la composición demográfica de los usuarios (distribución por género y representatividad geográfica) y el comportamiento del rango temporal de las escuchas.

Esta etapa nos permite identificar la representatividad de la muestra, el alcance global del dataset y la presencia de sesgos o valores anómalos (como fechas fuera de rango) antes de proceder con las reglas de limpieza en la Sección 3.

A. Consultas de Cobertura y Demografía

In [ ]:
```
# --- 1. Cobertura del catálogo de artistas ---
# De las 351,4M de escuchas, verificamos cuántos artistas distintos del c
q("SELECT approx_count_distinct(artist_id) AS artistas_escuchados_aprox F
```

Out[ ]:

	artistas_escuchados_aprox
0	318190

La tabla de artistas incluye los 3,19 millones de artistas del dataset original, pero solo unos 370 mil (11,6%) aparecen en las escuchas analizadas. Esto se debe a que el dataset conserva solo las canciones que pudieron encontrarse en Spotify. Como consecuencia, los datos representan principalmente a artistas conocidos/populares, y los artistas

locales o independientes quedan subrepresentados.

In [6]:
```
# Qué valores toma cada columna de texto y cuántas veces aparece cada uno
# (equivalente a value_counts() de pandas)

# Género
display(q("SELECT gender, COUNT(*) AS usuarios FROM users GROUP BY gender

# Países: cuántos hay en total y los 20 con más usuarios
display(q("SELECT COUNT(DISTINCT country) AS cantidad_paises FROM users")
display(q("SELECT country, COUNT(*) AS usuarios FROM users GROUP BY count
```

	gender	usuarios
0	m	36506
1	f	13937
2	n	4663
3		84

	cantidad_paises
0	208

	country	usuarios
0	US	10255
1	RU	5024
2	DE	4578
3	UK	4534
4	PL	4408
5	BR	3886
6	FI	1409
7	NL	1375
8	ES	1243
9	SE	1231
10	UA	1143
11	CA	1077
12	FR	1055
13	AU	976
14	IT	974
15	JP	806
16	NO	750
17	MX	705
18	CZ	632
19	BY	558

B. Consultas de Cobertura Temporal

In [7]:
```
# Primera y última escucha, convertidas de unix time a fecha legible
q("""
    SELECT to_timestamp(MIN("timestamp")) AS primera_escucha,
           to_timestamp(MAX("timestamp")) AS ultima_escucha
    FROM events
""")
```

Out[7]:

	primera_escucha	ultima_escucha
0	1970-01-01 00:00:36+00:00	2014-09-01 23:56:08+00:00

In [8]:
```
# Cantidad de escuchas por año.
# year() saca el año de la fecha; así vemos si hay años "imposibles" (ant
# y cuántas escuchas tienen.
por_anio = q("""
    SELECT year(to_timestamp("timestamp")) AS anio,
           COUNT(*) AS escuchas
    FROM events
    GROUP BY anio
    ORDER BY anio
```

```
""")
por_anio
```

Out[8]:

	anio	escuchas
0	1970	1
1	2005	786963
2	2006	2870944
3	2007	7111575
4	2008	12657501
5	2009	21707665
6	2010	34585255
7	2011	55158881
8	2012	90705031
9	2013	91813023
10	2014	34072494

2.6 Síntesis de la abstracción y características del dataset

A modo de síntesis de la etapa de exploración y abstracción de datos, se presentan las estructuras principales, tipos de variables y particularidades detectadas en las 6 tablas.

1. Tablas, granularidad y dimensiones

Tabla	Cada fila representa...	Filas	Clave de relación
events	Una escucha individual (un usuario reprodujo una canción en un instante)	351.469.333	`user_id` , `artist_id` , `track_id` , `timestamp`
users	Un perfil de usuario de Last.fm	55.190	`user_id` , `country`
artists	Un artista del catálogo LFM-1b completo	3.190.371	`artist_id`
acoustic_features	Una canción con sus descriptores de audio de Spotify	3.471.884	`track_id`
hofstede	Un país con sus dimensiones culturales	47	`ctr` (ISO-2)
happiness	Un país en un año específico de medición (WHR)	1.562	`country` , `year`

2. Relaciones y modelo de unión

- **events** actúa como la tabla central de hechos y se une directamente con **users** (`user_id`), **artists** (`artist_id`) y **acoustic_features** (`track_id`).

- **`users`** se une con **`hofstede`** mediante el código de país de 2 letras (`country = ctr`).
- **`happiness`** utiliza el nombre del país en inglés y el año (`country + year`), requiriendo mapeo de nombres de países para cruzarse con `users` / `hofstede` .

3. Clasificación lógica de atributos

Atributo	Tipo / Escala Lógica	Observaciones
`timestamp` , `registered_unixtime`	Temporal	Formato Unix Epoch Time (segundos en UTC)
`country` , `gender` , `artist_name`	Categórico (Nominal)	Textos y códigos ISO
`age` , `playcount`	Cuantitativo (Discreto)	`age` contiene valores faltantes codificados como `-1`
`danceability` , `energy` , `valence` , `acousticness` , `instrumentalness` , `speechiness` , `liveness` , `tempo` , `loudness`	Cuantitativo (Continuo)	Escalas normalizadas [0.0, 1.0], dB o BPM
`key` , `mode`	Categórico / Notación	Guardados como decimales (`DOUBLE`). `key` es cíclico (0–11) y `mode` es binario (0/1)
`Life Ladder` , dimensiones de Hofstede	Cuantitativo	Índices socioeconómicos y culturales [0, 10] y [0, 100]
`user_id` , `artist_id` , `album_id` , `track_id`	Identificadores	Claves primarias y foráneas para los JOINs

4. Observaciones y particularidades detectadas

- **Cobertura:** de los 3,19 millones de artistas de la tabla `artists` , aproximadamente 370.549 aparecen en las escuchas.
- **Países:** los usuarios pertenecen a 208 países, con una distribución muy concentrada: Estados Unidos (10.255), Rusia (5.024) y Alemania (4.578) son los que más usuarios tienen.Argentina tiene 282 usuarios.
- **Género:** además de "m" y "f", aparece el valor "n" (4663 veces), no documentado en el README.
- **Período:** el período declarado es enero 2005 – agosto 2014, pero la primera escucha registrada tiene fecha 01/01/1970 y la última 01/09/2014. Es una timestamp inválido.
- **events:** un mismo usuario puede escuchar la misma canción en loop. Las filas no están ordenadas por fecha.
- **users:** hay usuarios registrados en 2002–2003, antes del período de escuchas.
- **acoustic_features:** `key` y `mode` están guardados como decimales, pero son variables categóricas.
- **hofstede:** el código de país coincide con el de `users` . Se detectó un error de tipeo ("Bulgeria")

- **happiness:** no todos los países tienen datos desde el mismo año (Afganistán, desde 2008). Hay valores faltantes en las columnas de GINI.

3. Calidad de los datos y estrategias de limpieza

En esta sección se audita la integridad, consistencia y completitud del conjunto de datos unificado. Siguiendo la metodología de preparación de datos, no se busca únicamente identificar valores faltantes o fuera de rango, sino **evaluar su impacto sobre los análisis posteriores y definir reglas explícitas de tratamiento** (filtrado, imputación o conservación) para garantizar la validez del informe.

El análisis de calidad abarca cinco dimensiones críticas:

- **3.1 Valores nulos explícitos:** Detección de celdas sin información en cada entidad.
- **3.2 Faltantes "escondidos" y valores no documentados:** Cadenas vacías, códigos no declarados o banderas por defecto de las API.
- **3.3 Valores fuera de rango y atípicos (Outliers):** Edades inconsistentes, marcas temporales erróneas y comportamientos anómalos (p. ej., presencia de bots).
- **3.4 Verificación de duplicados:** Confirmación de unicidad en claves primarias y registros redundantes.
- **3.5 Integridad referencial entre tablas:** Auditoría de cobertura en los cruces (*JOINs*) del modelo.
- **3.6 Síntesis de decisiones de transformación y limpieza**

3. Calidad de los datos

- **3.1** valores nulos,
- **3.2** valores faltantes "escondidos" (vacíos o no documentados),
- **3.3** valores fuera de rango,
- **3.4** duplicados,
- **3.5** claves que no cruzan entre tablas.

3.1 Valores nulos explícitos

In [25]:
```
#3.1 valores nulos
# COUNT(*) cuenta todas las filas y COUNT(columna) solo las que NO son nu
# así que la diferencia es la cantidad de nulos de esa columna.
def contar_nulos(tabla):
    columnas = q(f"DESCRIBE {tabla}")["column_name"]
    partes = [f'COUNT(*) - COUNT("{col}") AS "{col}"' for col in columnas
    resultado = q(f"SELECT {', '.join(partes)} FROM {tabla}").T
    resultado.columns = ["nulos"]
    return resultado

for tabla in ["users", "artists","acoustic_features", "hofstede", "happin
    print(f"--- {tabla} ---")
    display(contar_nulos(tabla))
```

```
--- users ---
```

	nulos
user_id	0
country	0
age	0
gender	0
playcount	0
registered_unixtime	0

```
--- artists ---
```

	nulos
artist_id	0
artist_name	0
gender_counts	0

```
--- acoustic_features ---
```

	nulos
track_id	0
danceability	2655
energy	373
key	294
loudness	0
mode	294
speechiness	2654
acousticness	401
instrumentalness	454
liveness	759
valence	3595
tempo	0

```
--- hofstede ---
```

	nulos
no	0
ctr	0
country	0
power_distance	0
individualism	0
masculinity	0
uncertainty_avoidance	0
long_term_orientation	0
indulgence	0

```
--- happiness ---
```

	nulos
country	0
year	0
Life Ladder	0
Log GDP per capita	27
Social support	13
Healthy life expectancy at birth	9
Freedom to make life choices	29
Generosity	80
Perceptions of corruption	90
Positive affect	18
Negative affect	12
Confidence in national government	161
Democratic Quality	171
Delivery Quality	171
Standard deviation of ladder by country-year	0
Standard deviation/Mean of ladder by country-year	0
GINI index (World Bank estimate)	979
GINI index (World Bank estimate), average 2000-15	176
gini of household income reported in Gallup, by wp5-year	357

In [10]:
```
contar_nulos("events") #los eventos son millones
```

Out[10]:

	nulos
user_id	0
artist_id	0
album_id	0
track_id	0
timestamp	0

Observaciones "3.1 Valores nulos explícitos"

- **`events` , `artists` y `users` :** Cero valores nulos en todas sus columnas. La clave transaccional y los perfiles principales poseen 100% de completitud.
- **`acoustic_features` :** Presenta un nivel extremadamente bajo de nulos (<0.1% del total). La variable con más ausentes es `valence` (3.595 nulos sobre 3,47M de canciones). De acuerdo con la documentación de la API de Spotify, esto se debe a fallas en la extracción automática del DSP.
- **`hofstede` :** Cero nulos.
- **`happiness` :** Muestra nulos en los índices secundarios socioeconómicos (p. ej., `GINI index` con 979 nulos).

3.2 Faltantes "escondidos" y valores no documentados

In [11]:
```
# 3.2 Faltantes escondidos en users
# - País vacío: un texto '' no cuenta como nulo, pero tampoco es un país.
# - Género 'n': el README dice que solo puede ser 'm' o 'f', pero hay una
# - Género con otro valor
q("""
    SELECT
        COUNT(*) FILTER (WHERE TRIM(country) = '')            AS pais_vac
        COUNT(*) FILTER (WHERE gender = 'n')                  AS genero_n
        COUNT(*) FILTER (WHERE gender NOT IN ('m', 'f', 'n')) AS genero_o
        COUNT(*)                                              AS total_us
    FROM users
""")
```

Out[11]:

	pais_vacio	genero_n	genero_otro_valor	total_usuarios
0	0	4663	84	55190

In [12]:
```
# Todos los valores de género y cuántos usuarios tiene cada uno
q("SELECT gender, COUNT(*) AS usuarios FROM users GROUP BY gender ORDER B
```

Out[12]:

	gender	usuarios
0	m	36506
1	f	13937
2	n	4663
3		84

Observaciones "3.2 Faltantes "escondidos" y valores no documentados"

- **`users.gender`:** Además de las etiquetas válidas `'m'` y `'f'`, se detectaron **4.663 usuarios con el valor `'n'`** (no especificado/neutro) y **84 cadenas vacías**.
- **`users.country`:** Se identificaron cadenas vacías que representan usuarios sin localización.
- **`acoustic_features`:** Se registraron 209 canciones con `tempo = 0` y 2.651 pistas con `loudness = -60.0 dB`. Corresponden a archivos de audio silenciosos o con errores de procesamiento en Spotify.

3.3 Valores fuera de rango y anomalías temporales/ operativas

In [13]:
```
#3.3.a valores fuera de rango

# se consideran válidas las edades entre 10 y 90 años.
# Esta consulta muestra qué outliers aparecen y cuántos usuarios tiene ca
q("""
    SELECT age, COUNT(*) AS usuarios
    FROM users
    WHERE age < 10 OR age > 90
    GROUP BY age
    ORDER BY usuarios DESC
""")
```

Out[13]:

	age	usuarios
0	-1	12139
1	0	29
2	111	29
3	2	26
4	3	25
5	112	24
6	1	21
7	6	17
8	4	16
9	101	14
10	100	12
11	109	10
12	102	10
13	104	10
14	103	9
15	5	8
16	110	8
17	8	7
18	107	7
19	108	6
20	91	6
21	95	6
22	7	5
23	92	5
24	93	5
25	94	5
26	106	5
27	99	4
28	97	4
29	105	3
30	9	3
31	98	2
32	96	1

In [14]:
```
#3.3.b audio features
# Un tempo de 0 BPM o un volumen de -60 dB (silencio total) no son posibl
```

```
# son valores que Spotify no pudo calcular y quedaron como 0 o como el mí
q("""
    SELECT
        COUNT(*) FILTER (WHERE tempo = 0)      AS tempo_cero,
        COUNT(*) FILTER (WHERE loudness = -60) AS loudness_silencio,
        COUNT(*)                               AS total_canciones
    FROM acoustic_features
""")
```

Out[14]:

	tempo_cero	loudness_silencio	total_canciones
0	2651	209	3471884

In [15]:
```
#3.3.c Fechas

# tabla por anio calculada en la exploracion de datos
# Nos quedamos con los años fuera del período 2005-2014 y calculamos qué
fuera = por_anio[(por_anio["anio"] < 2005) | (por_anio["anio"] > 2014)]

print("Escuchas fuera de 2005-2014:", fuera["escuchas"].sum())
print("Porcentaje del total:", round(fuera["escuchas"].sum() / por_anio["
fuera
```

```
Escuchas fuera de 2005-2014: 1
Porcentaje del total: 0.0 %
```

Out[15]:

	anio	escuchas
0	1970	1

In [16]:
```
#3.3.d Valores atípicos: usuarios con actividad extrema

# Escuchas por usuario y por día.
# date_trunc('day', ...) recorta la fecha al día (sin hora), para contar
# Después nos quedamos con el máximo de cada usuario: su "día más activo"
por_usuario_dia = q("""
    SELECT user_id, MAX(escuchas_dia) AS max_escuchas_en_un_dia
    FROM (
        SELECT user_id,
               date_trunc('day', to_timestamp("timestamp")) AS dia,
               COUNT(*) AS escuchas_dia
        FROM events
        GROUP BY user_id, dia
    )
    GROUP BY user_id
""")

# describe() muestra mínimo, máximo, promedio y percentiles: así vemos si
por_usuario_dia["max_escuchas_en_un_dia"].describe(percentiles=[0.5, 0.9,
```

Out[16]:
```
count    55143.000000
mean       102.233230
std         93.336707
min          1.000000
50%         85.000000
90%        185.000000
99%        399.000000
max       3534.000000
Name: max_escuchas_en_un_dia, dtype: float64
```

In [17]:
```
# Cuántos usuarios superan las 500 escuchas en un mismo día (unas 33 hora
(por_usuario_dia["max_escuchas_en_un_dia"] > 500).sum()
```

Out[17]: np.int64(307)

- La mayoría de los usuarios tiene un máximo diario razonable (mediana: 85 escuchas en su día más activo; 99% por debajo de 399). Sin embargo, 307 usuarios superan las 500 escuchas en un mismo día, con un máximo de 3.534. Estos valores no corresponden podrian ser bots o de importaciones masivas de escuchas.
- 47 usuarios no tienen ninguna escucha en el dataset (55.143 usuarios con escuchas sobre 55.190).

aclaracion de que es lo que Last.fm considera 1 escucha: si dura mas de 30 segundos o si se escucho al menos la mitad. fuente: https://www.last.fm/api/scrobbling

In [18]:
```
# Umbral basado en la regla de Last.fm: cada escucha requiere al menos la
# Con canciones de ~4 minutos, son al menos 2 minutos por escucha: máximo
(por_usuario_dia["max_escuchas_en_un_dia"] > 720).sum()
```

Out[18]: np.int64(140)

In [19]:
```
# Días en los que un usuario superó las 720 escuchas, y cuántas escuchas
q("""
    SELECT COUNT(*)          AS dias_anomalos,
           SUM(escuchas_dia) AS escuchas_en_dias_anomalos
    FROM (
        SELECT user_id,
               date_trunc('day', to_timestamp("timestamp")) AS dia,
               COUNT(*) AS escuchas_dia
        FROM events
        GROUP BY user_id, dia
    )
    WHERE escuchas_dia > 720
""")
```

Out[19]:

	dias_anomalos	escuchas_en_dias_anomalos
0	664	727379.0

- Esos 140 usuarios acumulan 664 días anómalos, que suman 727.379 escuchas (0,21% del total), con un promedio de 1.095 escuchas por día. Se eliminarán las escuchas de esos días, conservando el resto de la actividad de los usuarios: la pérdida de datos es mínima y evita que estos registros distorsionen el análisis temporal.

In [20]:
```
#3.3.e consistencia de cuantas escuchas hay antes de haberse registrado a
#puede ser que la fecha este mal

# Cantidad de escuchas con fecha anterior a la fecha de registro del usua
q("""
    SELECT COUNT(*) AS escuchas_antes_del_registro,
           COUNT(DISTINCT e.user_id) AS usuarios_afectados
    FROM events e
```

```
    JOIN users u ON e.user_id = u.user_id
    WHERE e."timestamp" < u.registered_unixtime
""")
```

Out[20]:

	escuchas_antes_del_registro	usuarios_afectados
0	2963	760

Observaciones "3.3 Valores fuera de rango y anomalías temporales/operativas"

- **Edades extremas (`users.age`):** Existen 12.139 usuarios con `age = -1` (código utilizado para valores no declarados)[cite: 18]. Asimismo, figuran edades atípicas de hasta 112 años. Se define el rango biológicamente y éticamente válido en **[10, 90] años**.
- **Timestamps inválidos (`events`):** Se detectó 1 registro con `timestamp = 0` (correspondiente a `1970-01-01 00:00:36 UTC`), fuera del rango formal del dataset (2005–2014).
- **Actividad masiva / Presencia de Bots (`scrobbling` anómalo):** Se analizó la actividad diaria por usuario. Last.fm requiere que una canción se escuche al menos 2 minutos para contarse como reproducción. Esto impone un máximo físico de ~720 escuchas por día (24 horas contiguas). Se detectaron **140 usuarios que superan las 720 escuchas/día en 664 jornadas anómalas**, acumulando 727.379 reproducciones (0,21% del total) con promedios de 1.095 escuchas/día.

3.4 Verificación de duplicados

In [21]:

```
# 3.4 Duplicados
# COUNT(*) cuenta todas las filas; COUNT(DISTINCT ...) cuenta los IDs sin
# Si los dos números dan igual, no hay IDs duplicados.
q("""
    SELECT
        (SELECT COUNT(*) FROM users)                              AS filas
        (SELECT COUNT(DISTINCT user_id) FROM users)               AS usuar
        (SELECT COUNT(*) FROM acoustic_features)                  AS filas
        (SELECT COUNT(DISTINCT track_id) FROM acoustic_features) AS canci
        (SELECT COUNT(*) FROM artists)                            AS filas
        (SELECT COUNT(DISTINCT artist_id) FROM artists)           AS artis
""")
```

Out[21]:

	filas_users	usuarios_distintos	filas_canciones	canciones_distintas	filas_artistas	art
0	55190	55190	3471884	3471884	3190371	

In [22]:

```
# Escuchas duplicadas de verdad: mismo usuario, misma canción y mismo tim
# Agrupamos por esas tres columnas y contamos los grupos que aparecen más
q("""
    SELECT COUNT(*) AS grupos_duplicados, SUM(veces - 1) AS filas_sobrant
    FROM (
        SELECT user_id, track_id, "timestamp", COUNT(*) AS veces
        FROM events
        GROUP BY user_id, track_id, "timestamp"
        HAVING COUNT(*) > 1
    )
```

```
""")
```

Out[22]:

	grupos_duplicados	filas_sobrantes
0	0	NaN

Observaciones "3.4 Verificación de duplicados"

- **Claves primarias:** Se confirmó la ausencia de IDs duplicados en `users.user_id`, `artists.artist_id` y `acoustic_features.track_id`.
- **Eventos redundantes:** La búsqueda de registros idénticos en la tupla (`user_id`, `track_id`, `timestamp`) arrojó **0 filas duplicadas**.

3.5 Integridad referencial y cruces entre tablas

In [23]:
```
#3.5.a escuchas sin usuario, artista o cancion

# LEFT JOIN conserva TODAS las escuchas, aunque no hagan match en la otra
# Si no la encuentran, las columnas de la otra tabla quedan en NULL.
# Contamos cuántas escuchas quedaron sin match en cada tabla, (ideal 0).
q("""
    SELECT
        COUNT(*) FILTER (WHERE u.user_id   IS NULL) AS escuchas_sin_usuar
        COUNT(*) FILTER (WHERE a.artist_id IS NULL) AS escuchas_sin_artis
        COUNT(*) FILTER (WHERE f.track_id  IS NULL) AS escuchas_sin_audio
    FROM events e
    LEFT JOIN users u             ON e.user_id   = u.user_id
    LEFT JOIN artists a           ON e.artist_id = a.artist_id
    LEFT JOIN acoustic_features f ON e.track_id  = f.track_id
""")
```

Out[23]:

	escuchas_sin_usuario	escuchas_sin_artista	escuchas_sin_audio_features
0	0	0	0

In [24]:
```
#3.5.b paises de users en hoefstede

# Ambas tablas usan el código de país de 2 letras (users.country y hofste
# Primero vemos cómo figura el Reino Unido en hofstede: en users aparece
display(q("SELECT ctr, country FROM hofstede WHERE ctr IN ('UK', 'GB')"))

# Después contamos cuántos usuarios y países quedan sin datos de Hofstede
q("""
    SELECT COUNT(*)                  AS usuarios_sin_hofstede,
           COUNT(DISTINCT u.country) AS paises_sin_hofstede
    FROM users u
    LEFT JOIN hofstede h ON u.country = h.ctr
    WHERE h.ctr IS NULL
""")
```

	ctr	country
0	UK	Great Britain

Out[24]:

	usuarios_sin_hofstede	paises_sin_hofstede
0	1881	161

Observaciones "3.5 Integridad referencial y cruces entre tablas"

- **Eventos huérfanos:** El 100% de los eventos en `events` encuentra coincidencia en `users`, `artists` y `acoustic_features`.
- **Cruce cultural (`users <-> hofstede`):** La tabla `hofstede` cubre 47 países. Aunque 161 países pequeños no tienen datos culturales, los 47 disponibles concentran a **53.309 de los 55.190 usuarios (>96,5% de cobertura global)**.
- **Discrepancia ISO-2:** Se identificó que Reino Unido figura como `'UK'` en `users` y `hofstede`, debiendo convertirse a `'GB'` (estándar ISO-2) para su correcto mapeo geoespacial en mapas.

3.6 Síntesis de decisiones de transformación y limpieza

A partir del diagnóstico de calidad, se establecen las siguientes reglas de negocio que se aplicarán para generar las vistas y datasets limpios (`/data/clean/`):

1. **Filtrado de fechas inválidas:** Se elimina la escucha del año 1970.
2. **Tratamiento de Bots:** Se descartan exclusivamente los **664 días anómalos** donde la actividad supere las 720 escuchas/día. Esto remueve el sesgo de automatización (0,21% de los datos) sin perder a los usuarios ni su actividad legítima.
3. **Tratamiento de variables demográficas:** En el análisis de edad se considerarán exclusivamente los valores en el rango $10 \leq \text{age} \leq 90$, tratando el resto como `NaN`. Las categorías `'n'` y vacíos en género se mantendrán clasificadas como `"No especificado"`.
4. **Estandarización ISO-2:** Se aplica el remapeo explícito `'UK'` $\rightarrow$ `'GB'` en la columna país para asegurar la integración transparente con herramientas de visualización geográfica.